

Data Structures Interview Questions

Բոլոր հնարավոր DS հարցազրույցի հարցերը (C++)

Այս ֆայլը Data Structures-ի interview հարցերի ամբողջական ժողովածու է: Երեք մաս.

Մաս A — 40 multiple-choice հարց (4 տարբերակ, մեկը ճիշտ):

Մաս B — 40 բաց (open) տեսական հարց՝ առանց տարբերակների:

Մաս C — 20 tricky կոդային/complexity հարց:

Բոլոր պատասխանները **միայն վերջի էջերում** են (Answer Key A, B, C): Առաջ նայելու առաջ փորձիր ինքդ պատասխանել:

ՄԱՍ A — Multiple Choice (40)

Q1

Array-ի element-ի index-ov mutqy inch complexity e? - (a) $O(1)$ - (b) $O(n)$ - (c) $O(\log n)$ - (d) $O(n^2)$

Q2

Array-um memory-n inchpes e pahvum? - (a) scattered - (b) contiguous (koxq-koxqi) - (c) tree-i pes - (d) random

Q3

Linked list-um element-i access-y (index-ov) inch e? - (a) $O(1)$ - (b) $O(n)$ - (c) $O(\log n)$ - (d) $O(1)$ average

Q4

Singly linked list-i node-y inch uni? - (a) miayn data - (b) data + next pointer - (c) data + next + prev - (d) miayn pointer

Q5

Doubly linked list-i node-y inch uni? - (a) data + next - (b) data + next + prev - (c) miayn data - (d) 2 data

Q6

Stack-y inch principov e ashxatum? - (a) FIFO - (b) LIFO - (c) sorted - (d) random

Q7

Queue-n inch principov e ashxatum? - (a) LIFO - (b) FIFO - (c) sorted - (d) random

Q8

Stack-i push/pop complexity? - (a) $O(n)$ - (b) $O(\log n)$ - (c) $O(1)$ - (d) $O(n^2)$

Q9

Vory DS-y ka function call-eri hamar (call stack)? - (a) queue - (b) stack - (c) heap - (d) tree

Q10

BFS-i hamar vory DS-n e ogtagorcvm? - (a) stack - (b) queue - (c) heap - (d) trie

Q11

DFS-i hamar sovorabar vory DS-n e? - (a) queue - (b) stack / recursion - (c) heap - (d) hash

Q12

Hash table-i mijin lookup complexity? - (a) $O(\log n)$ - (b) $O(1)$ average - (c) $O(n)$ always - (d) $O(n^2)$

Q13

Hash table-i worst case complexity? - (a) $O(1)$ - (b) $O(\log n)$ - (c) $O(n)$ - (d) $O(n^2)$

Q14

Inch e collision-y hash table-um? - (a) key jnjum - (b) erku key nuyn bucket/hash - (c) memory leak - (d) sorted karg

Q15

Balanced BST-i search complexity? - (a) $O(1)$ - (b) $O(\log n)$ - (c) $O(n)$ - (d) $O(n \log n)$

Q16

BST-i inorder traversal-y inch karg e talis? - (a) random - (b) sorted (ac) - (c) reverse sorted - (d) level order

Q17

Unbalanced BST-i worst case? - (a) $O(1)$ - (b) $O(\log n)$ - (c) $O(n)$ - (d) $O(n^2)$

Q18

C++ `std::map` -y inch tree e ogtagorcvm? - (a) AVL - (b) Red-Black Tree - (c) B-Tree - (d) Trie

Q19

Max-heap-um root-y inch e? - (a) amenapokry - (b) amenamecy - (c) mijiny - (d) random

Q20

Heap-i insert (push) complexity? - (a) $O(1)$ - (b) $O(\log n)$ - (c) $O(n)$ - (d) $O(n \log n)$

Q21

Heap-i top (peek) complexity? - (a) $O(1)$ - (b) $O(\log n)$ - (c) $O(n)$ - (d) $O(n \log n)$

Q22

C++ `priority_queue` -n default-ov inch heap e? - (a) min-heap - (b) max-heap - (c) balanced - (d) sorted

Q23

Binary tree-i inorder = ? - (a) Root Left Right - (b) Left Root Right - (c) Left Right Root - (d) Right Root Left

Q24

Preorder = ? - (a) Root Left Right - (b) Left Root Right - (c) Left Right Root - (d) level order

Q25

Postorder = ? - (a) Root Left Right - (b) Left Root Right - (c) Left Right Root - (d) Right Left Root

Q26

Level-order traversal-y inch DS-ov e? - (a) stack - (b) queue - (c) heap - (d) recursion

Q27

Trie-i search/insert complexity (L = ba'ri erkarutyun)? - (a) $O(1)$ - (b) $O(L)$ - (c) $O(n)$ - (d) $O(\log n)$

Q28

Trie-n inchi hamar e amenaharmar? - (a) sorting - (b) prefix search / autocomplete - (c) shortest path - (d) frequency

Q29

Graph-i adjacency list-i space complexity? - (a) $O(V^2)$ - (b) $O(V + E)$ - (c) $O(E^2)$ - (d) $O(1)$

Q30

Adjacency matrix-i space complexity? - (a) $O(V + E)$ - (b) $O(V^2)$ - (c) $O(E)$ - (d) $O(1)$

Q31

BFS/DFS complexity (adjacency list)? - (a) $O(V^2)$ - (b) $O(V + E)$ - (c) $O(V * E)$ - (d) $O(E^2)$

Q32

Array-i mejtexum insert complexity? - (a) $O(1)$ - (b) $O(\log n)$ - (c) $O(n)$ - (d) $O(n^2)$

Q33

Linked list-i head-um insert? - (a) $O(1)$ - (b) $O(n)$ - (c) $O(\log n)$ - (d) $O(n^2)$

Q34

Vory DS-n e amenalav frequency count-i hamar? - (a) array - (b) unordered_map (hash) - (c) stack - (d) queue

Q35

Vory DS-n e petq sorted karg + range query? - (a) unordered_map - (b) map / set (BST) - (c) stack - (d) queue

Q36

Top-K amenamec element-neri hamar? - (a) stack - (b) heap (priority_queue) - (c) queue - (d) trie

Q37

Vory traversal-n e talis BST sorted? - (a) preorder - (b) inorder - (c) postorder - (d) level

Q38

Cache locality-i taxatsanqov vory DS-n e aveli arag? - (a) linked list - (b) array/vector (contiguous) - (c) tree - (d) hash

Q39

Stack-ov inch problem e klasik lucvum? - (a) shortest path - (b) balanced parentheses - (c) frequency - (d) sorting

Q40

Dijkstra-i shortest path-i hamar vory DS-n e ogtagorcvm? - (a) stack - (b) min-heap (priority_queue) - (c) trie - (d) hash only

ՄԱՍ B — Open Questions (40)

Բաց հարաբերակցություն, փոքրիկ ամբողջական տարածք: Մոդելային տարածքներ` Answer Key B-um.

1. Ի՞նչ է data structure-ը, ինչպես է ներդրումը օգտագործել.
2. Array vs Linked List — մեկտեսն տարբերություն.
3. Ինչու array-ի access-ը $O(1)$ է, իսկ linked list-ին $O(n)$.
4. Singly vs Doubly linked list.
5. Ինչպես է reverse անում linked list (աճյուն կոդ, տրամադրություն).
6. Stack-ը ինչ է, LIFO-ն ետևի, ինչ խնդիր է լուծել.
7. Queue-ն ինչ է, FIFO-ն ետևի, ուր է օգտագործվում.
8. Stack vs Queue տարբերություն.
9. Ինչ է circular queue / deque.
10. Hash table ինչպես է աշխատում (hash function, bucket).
11. Ինչ է collision-ը, ինչպես է լուծվում (chaining, open addressing).
12. Ինչու hash table-ի worst case-ը $O(n)$ է.
13. Ինչ է load factor / rehashing.
14. Tree vs Graph տարբերություն.
15. Binary tree vs Binary Search Tree.
16. BST-ի inorder traversal ինչու է սորտավորված.
17. Ինչու BST-ը կարող է $O(n)$ ժամկետով լուծել.
18. Ինչ է AVL / Red-Black Tree.
19. Traversal-ների տեսակներ (inorder/preorder/postorder/level).
20. Ինչ է heap-ը, max vs min heap.
21. Heap-ի insert/extract ինչպես է աշխատում (heapify).
22. Priority queue ինչի համար է, ուր ինչ է.
23. Ինչ է complete binary tree.
24. Graph-ի ներկայացման ձևեր (adjacency list vs matrix).
25. BFS vs DFS — ուր է օգտագործվում.
26. Directed vs Undirected, Weighted vs Unweighted graph.
27. Ինչ է cycle-ը graph-ում, ինչպես է հայտնաբերվում.
28. Ինչ է topological sort.
29. Trie-ն ինչ է, ինչի համար է.
30. Ինչ է connected component.
31. Երբ է օգտագործվում hash table vs BST.
32. Երբ է օգտագործվում array vs linked list.
33. Ինչ է amortized complexity (vector push_back).
34. Ինչ է in-place algorithm.
35. Ինչ է stable data structure/sort.

36. Inch e space vs time tradeoff, ber orinak.
 37. Inchpes es haytnaberum cycle linked list-um (Floyd).
 38. Inch e LRU cache, inch DS-erov e sarqvum.
 39. Stack-ov inchpes es implement anum queue (kam hakaraky).
 40. Vory DS-n es entrum: fast lookup + sorted iteration erku-si hamar?
-

UUU C — Tricky Code / Complexity (20)

C1

```
vector<int> v = {1,2,3};  
v.insert(v.begin(), 0);
```

Inch complexity e ays insert-y? - (a) $O(1)$ - (b) $O(n)$ (bolor element-ner sharzum) - (c) $O(\log n)$ - (d) CE

C2

```
stack<int> st;  
st.push(1); st.push(2); st.push(3);  
st.pop();  
cout << st.top();
```

- (a) 1
- (b) 2
- (c) 3
- (d) CE

C3

```
queue<int> q;  
q.push(1); q.push(2); q.push(3);  
q.pop();  
cout << q.front();
```

- (a) 1
- (b) 2
- (c) 3
- (d) CE

C4

```
priority_queue<int> pq;  
pq.push(3); pq.push(1); pq.push(5);  
cout << pq.top();
```

- (a) 1
- (b) 3
- (c) 5
- (d) 9

C5

```
priority_queue<int, vector<int>, greater<int>> pq;  
pq.push(3); pq.push(1); pq.push(5);  
cout << pq.top();
```

- (a) 1
- (b) 3
- (c) 5
- (d) CE

C6

BST-i inorder traversal, tree:

```
    5  
   / \  
  3   8  
 / \  
1   4
```

Inch karg e tpvum inorder-ov? - (a) 5 3 8 1 4 - (b) 1 3 4 5 8 - (c) 1 4 3 8 5 - (d) 5 3 1 4 8

C7

Nuyn tree-i preorder? - (a) 5 3 1 4 8 - (b) 1 3 4 5 8 - (c) 1 4 3 8 5 - (d) 5 8 3 4 1

C8

```
unordered_map<int,int> m;  
m[5]++;  
cout << m[5];
```

- (a) 0
- (b) 1
- (c) undefined (garbage)
- (d) CE

C9

```
map<int,int> m;  
m[3]=1; m[1]=1; m[2]=1;  
for (auto& p : m) cout << p.first << " ";
```

- (a) 3 1 2
- (b) 1 2 3
- (c) random
- (d) CE

C10

```
unordered_map<int,int> m;  
m[3]=1; m[1]=1; m[2]=1;  
for (auto& p : m) cout << p.first << " ";
```

- (a) misht 1 2 3
- (b) misht 3 1 2
- (c) karg erashxavorvac che
- (d) CE

C11

Balanced BST-um n element, search complexity? - (a) $O(1)$ - (b) $O(\log n)$ - (c) $O(n)$ - (d) $O(n \log n)$

C12

Sorted array-i mej binary search:

```
vector<int> v = {1,3,5,7,9};  
binary_search(v.begin(), v.end(), 5);
```

- (a) $O(n)$
- (b) $O(\log n)$
- (c) $O(1)$
- (d) CE

C13

```
list<int> lst = {1,2,3};  
auto it = lst.begin();  
it + 2;
```

- (a) OK -> 3
- (b) compile error (list random access chuni)
- (c) $O(n)$
- (d) undefined

C14

```
vector<int> v = {1,2,3};  
auto it = v.begin();  
it + 2;  
cout << *it;
```

- (a) compile error
- (b) 1 (it inqy chi poxvum)
- (c) 3
- (d) undefined

C15

Heap-i (priority_queue) build heap complexity n element-ic? - (a) $O(n)$ - (b) $O(n \log n)$ - (c) $O(\log n)$ - (d) $O(n^2)$

C16

Graph adjacency matrix, edge (u,v) ka te che stugum? - (a) $O(V)$ - (b) $O(1)$ - (c) $O(E)$ - (d) $O(V+E)$

C17

```
stack<int> st;  
cout << st.top();    // dataark stack
```

- (a) 0
- (b) undefined behavior (empty stack top)
- (c) CE
- (d) -1

C18

Linked list-i mejtexy (middle) hasnelu complexity? - (a) $O(1)$ - (b) $O(n)$ - (c) $O(\log n)$ - (d) $O(n^2)$

C19

```
vector<int> v;  
v.reserve(100);  
cout << v.size() << " " << (v.capacity() >= 100);
```

- (a) 100 1
- (b) 0 1
- (c) 100 0
- (d) 0 0

C20

Trie-um "apple" bańi search complexity? - (a) $O(1)$ - (b) $O(5) = O(L)$ (bańi erkarutyun) - (c) $O(n)$ - (d) $O(\log n)$

ANSWER KEY A (Multiple Choice)

1-a · 2-b · 3-b · 4-b · 5-b · 6-b · 7-b · 8-c · 9-b · 10-b 11-b · 12-b · 13-c · 14-b · 15-b · 16-b · 17-c · 18-b ·
19-b · 20-b 21-a · 22-b · 23-b · 24-a · 25-c · 26-b · 27-b · 28-b · 29-b · 30-b 31-b · 32-c · 33-a · 34-b · 35-b
· 36-b · 37-b · 38-b · 39-b · 40-b

ANSWER KEY B (Open Questions)

1. DS-y tvyalneri kazmakerpman dzev e ardyunavet mutq/poxum/voronum-i hamar. Entrum es est` inch gorcoxutyunner en hachaxaki, complexity, memory. Chisht DS -> arag algoritm.
2. Array **contiguous memory, O(1) index access, O(n) insert/delete mejtexum, cache-friendly, fixed/dynamic. Linked list** scattered, O(n) access, O(1) insert/delete (pointer-ov), aveli shat memory (pointer-ner).
3. Array-i element-nery koxq-koxqi en -> hascee = base + i*size (uxix ha2vark). Linked list-i node-nery scattered en -> petq e qayl aʀ qayl gnal head-ic (index chka).
4. Singly **amen node data + next (miayn araj)**. Doubly **data + next + prev (araj u het)**. Doubly-n aveli shat memory bayc erku uxxutyamb sharzum.
5. (Tes veraki) Doubly` **data + 2 pointer (next, prev)**, tuyl e talis het sharzvel u O(1) delete tvyal node-i (prev-ov).
6. Stack **LIFO (Last In First Out)**, **verjin avelacracy araji hanvum. push/pop/top O(1)**. **Problem-ner** parentheses, undo/redo, DFS, function call stack, expression eval.
7. Queue **FIFO (First In First Out)**, **araji avelacracy araji hanvum. push (back)/pop (front) O(1)**. **Ogtagorcum** BFS, scheduling, buffering.
8. Stack LIFO (mek codic), Queue FIFO (erku codic` back-um add, front-ic remove).
9. Circular queue **array-based queue, vor verjic verɫunnum sksin (memory reuse)**. Deque double-ended queue, erku codic push/pop O(1).
10. hash(key) -> bucket index -> element pahvum ayd bucket-um. Lookup` hash hanıvel -> bucket gtneı -> O(1) average.
11. Collision **erku key nuyn bucket. Luzum** chaining (linked list bucket-um) kam open addressing (hajord azat bucket).
12. Shat collision kam vat hash function -> bucket-i list erkarum -> lookup O(n) darnum. Kam rehashing-i pahin.
13. Load factor = size/buckets. Gerazancelis (>~1) rehash` buckets x2, bolor element verabashxum (O(n) ayd pahin).
14. Tree **hierarchical, connected, acyclic, n node -> n-1 edge, mek root. Graph** vertices + edges, karox e cycle, disconnected, weighted lineı.
15. Binary tree **amen node <=2 child, kanon chka. BST** left < root < right (amen node-i hamar) -> sorted, search O(log n) balanced.
16. Inorder` Left-Root-Right. BST-i kanony (left<root<right) apahovum e, vor sa talis e acy sorted karg.

17. Sorted tıyal insert -> tree darning linked list (mek koxm) -> height $O(n)$ -> search $O(n)$.
Luzum` self-balancing (AVL, Red-Black Tree).
18. AVL xıst balanced BST (height diff ≤ 1). Red-Black moteqik balanced, aveli qich rotation insert-i zham (std::map/set-y RB-tree e).
19. inorder (L-Root-R), preorder (Root-L-R), postorder (L-R-Root), level-order (BFS, queue-ov, mak-mak).
20. Heap complete binary tree. Max-heap parent $\geq$ child (root max); min-heap` parent $\leq$ child (root min). Sovorabar array-ov e ıııpkayacvum.
21. Insert avelacnum verjum, heta "bubble up" (parent-i het poxanakum minchev kanon) -> $O(\log n)$. Extract root hanum, verjin element ver, "bubble down" (heapify) -> $O(\log n)$.
22. Priority queue misht amenamec/pokry element-y talis. Ners heap. push/pop $O(\log n)$, top $O(1)$. C++ default max-heap.
23. Complete binary tree` bolor level-nery lcvac, baci verjin-ic vory lcvum e dzaxic aj. Heap-y complete binary tree e.
24. Adjacency
list amen vertex-i harevanneri cucak, $O(V+E)$ space, sparse-i hamar. Adjacency matrix $V \times V$ table, $O(V^2)$, edge check $O(1)$, dense-i hamar.
25. BFS queue, level-by-level, shortest path unweighted, ognayin. DFS stack/recursion, xory, cycle/topo sort/components. Erkusn el $O(V+E)$.
26. Directed edge mek uxxutyamb ($A \rightarrow B$); undirected erku.
Weighted edge-y uni kshir (cost); unweighted bolory havasar.
27. Cycle uxi vor versıwınum e skizb. Undirected DFS + parent (visited harevan vor parent che -> cycle). Directed` recursion stack-um visited node.
28. Topological sort` DAG-i vertex-neri gcayin karg, vortex amen edge $u \rightarrow v$ -i hamar u-n galis e v-ic araj. Kahn's (in-degree + BFS) kam DFS.
29. Trie` prefix tree string-eri hamar, amen node mek char. insert/search/prefix $O(L)$. Autocomplete, spell check, dictionary.
30. Connected component` graph-i entaxumb, vortex amen vertex hasaneli e myusic. DFS/BFS-ov hamıvum en qanaky.
31. Hash (unordered_map) $O(1)$ avg lookup, kargy petq che, frequency/dedup. BST (map) $O(\log n)$, sorted, range query / lower_bound petq e.
32. Array random access, index, sorting, chapı hayni. Linked list hachaxaki insert/delete, chapı anhayt, iterator stability.
33. Amortized complexity mijin arjeqı shat gorcoxutyunneri vra. vector push_back sovoraky $O(1)$, reallocation-y $O(n)$, bayc x2 growth -> mijiny amortized $O(1)$.

34. In-

place **algorithm vor chi ogtagorcum lracucich (kam $O(1)$) memory, poxum e unix input-y (orinak reverse, bubble sort).**

35. Stable **pahpanum e havasar element-neri skzbnakan karg (sort-i depqum).** Stable sort merge, insertion; unstable ` quick, heap.

36. Space-time tradeoff ` aveli shat memory -> aveli arag (orinak hash table $O(1)$ lookup bayc $O(n)$ memory); kam qich memory -> aveli dandaghn.

37. Floyd's Tortoise & Hare ` erku pointer (slow 1 qayl, fast 2 qayl). Cycle-i depqum handipum en. $O(n)$ time, $O(1)$ space.

38. LRU cache **hash map + doubly linked list. Map** $O(1)$ lookup; list ` karg (verjercic hin). get/put $O(1)$.

39. Queue erku stack-ov ` mek in-stack (push), mek out-stack (pop); out dataark linelis in-y sharzum out (reverse). Amortized $O(1)$.

40. map (BST) -> fast ($O(\log n)$) lookup + sorted iteration; kam hash map + sorted structure. Ete erkusn el kritik ` map (sorted iteration hash-y chuni).

ANSWER KEY C (Tricky Code)

C1-b · C2-b · C3-b · C4-c · C5-a · C6-b · C7-a · C8-b · C9-b · C10-c · C11-b · C12-b · C13-b · C14-b · C15-a · C16-b · C17-b · C18-b · C19-b · C20-b

Bacatrutyunner (C)

C1 begin-um insert -> bolor element-ner mek qayl aj -> $O(n)$. **C2** push 1,2,3 -> pop (3 hanvum) -> top = 2. **C3** push 1,2,3 -> pop (1 hanvum, FIFO) -> front = 2. **C4** default max-heap -> top = 5 (amenamecy). **C5** greater -> min-heap -> top = 1. **C6** inorder (L-Root-R) BST -> sorted -> 1 3 4 5 8. **C7** preorder (Root-L-R) -> 5 3 1 4 8. **C8** m[5]++ -> chka key -> steghcum 0-ov, ++ -> 1. **C9** map sorted -> 1 2 3. **C10** unordered_map -> karg erashxavorvac che. **C11** balanced BST search -> $O(\log n)$. **C12** binary search sorted-um -> $O(\log n)$. **C13** list random access chuni -> it+2 compile error. **C14** it+2-y nor iterator e steghcum bayc chi veragrvm it-in -> it mnum begin -> $it=1$. **C15** build heap n element-ic -> $O(n)$ (ne $O(n \log n)$). **C16** adjacency matrix edge check -> $mat[u][v]$ -> $O(1)$. **C17** empty stack-i top() -> undefined behavior. **C18** linked list middle-i hasnel -> $O(n)$ (qayl ar qayl). **C19** reserve `size=0, capacity>=100 -> "0 1". **C20*** trie "apple" search -> $O(L) = O(5)$.

Endhanur kanonner (hisher interview-in)

Array: access $O(1)$, insert/delete mid $O(n)$, contiguous

List: access $O(n)$, insert/delete $O(1)$ w/ pointer, scattered

Stack: LIFO, push/pop/top $O(1)$

Queue: FIFO, push/pop/front $O(1)$

Hash: $O(1)$ avg, $O(n)$ worst, unsorted

BST: $O(\log n)$ balanced / $O(n)$ unbalanced, inorder=sorted

Heap: top $O(1)$, push/pop $O(\log n)$, build $O(n)$, default max

Trie: $O(L)$ operations, prefix

Graph: BFS(queue)/DFS(stack) $O(V+E)$, adj list (sparse)/matrix (dense $O(1)$ edge)

TRAVERSAL: inorder=LRootR (BST sorted) | preorder=RootLR | postorder=LRRoot | level=BFS

map=sorted ($O(\log n)$) | unordered_map=hash ($O(1)$ avg, no order)

empty stack/queue top/front -> UB